# Audits, Testing and Fact-Checking

The checks that stand between a change and a node, the tests that exercise the servers and the audio, the audits a node runs on itself, how the summaries in this series were checked against the code, and a plan for finding a problem.

CI CHECKS | TEST SUITES | NODE AUDITS | FIND A PROBLEM

SUMMARY · READ-ONLY REVIEW

Sources: ci/, .github/workflows/, channels/, the tool files, and the checks run for this summary on branch ccr-9d4fe19f-ssud83.

# Five places a problem can be caught

| LAYER | WHEN | WHAT IT IS |
|---|---|---|
| **1. Local checks** | Before a push | `ci/check_repo.py`, `py_compile` and `pyflakes` on changed Python, `node --check` on page script, `bash -n` on shell, and a test of the changed behaviour (CLAUDE.md) |
| **2. The `checks` workflow** | Every pull request | The same repo checks, plus the test suites in Part 3 |
| **3. The release gate** | Daily and on each push to main | The `promote-stable` workflow decides what reaches the Stable channel (Part 4) |
| **4. Run-time guards** | On a node, at the moment of a change | Confirmation, gates, locks and rollback in the tools (Part 5) |
| **5. Node audits and health readings** | On a node, any time | The SysMon audit, health verdicts, the boot report, the Suite Log (Part 5) |

The first three layers look at the code. The last two look at a running node. A problem found on a node should become a test or a check, so the next change cannot bring it back.

# What check_repo.py enforces

The script compares the changed files with the base branch and fails the pull request on any of these. It prints each problem, or "OK" with the number of files checked.

| GROUP | RULE |
|---|---|
| **Code** | Every changed Python file compiles and passes `pyflakes`. Page script inside a Python file is extracted and parsed with `node --check` (module scripts as modules). Shell files pass `bash -n`. |
| **Versions** | One file per tool. The file name and the version inside agree. Three parts for a release, four for a cosmetic build. The pi02w suffix is present or absent as the tool requires. A new version is newer than the previous file. A cosmetic build follows its release and may keep its file name only if the code did not change, only text. |
| **Records** | The README lists the file. The tool's changelog has an entry and its "Current file" line is right. |
| **Shared code** | Each function in `shared/confio.py` is identical in every tool its `USED_BY` table lists. The shared look (font and colour lines, welcome and licence text) is present in the eight web tools. The launcher text is identical wherever it is written. |
| **Channels** | `channels/stable.json` may not be edited by a pull request. The other channel files must have the right shape. |
| **Android** | A change under `android/` raises the app version and adds a changelog entry. |

The checks cannot judge whether a change is correct. They keep the files, versions and copies consistent, so a node can mix versions safely.

# What the tests exercise

| SUITE | WHAT IT PROVES | IN THE WORKFLOW | RESULT WHEN RUN FOR THIS SUMMARY |
|---|---|---|---|
| `ci/test_ci.py` | 241 tests: the repo checker, promotion rules, atomic writes, Confgetter and Confputter, Node Talker cards and behaviour, receive-health readings, the link and push-to-talk logs, listen events, and the Suite Log | Yes | 241 passed |
| `test_talker_tx_baseline.py` | Transmit audio reaches the USRP port at the expected level, compared with a stored baseline (`ci/fixtures`) | Yes | 2 passed |
| `test_noderecorder_raw.py` | Node Recorder's raw audio path | Yes | 3 passed |
| `test_talker_baseline.py` | Receive audio through the cleanup stage: tone levels, onset latency and signal cycles against a stored baseline | Yes | 3 passed (47 s) |
| `test_servers.py` | Eight servers (Dashboard, pi02w Dashboard, SysMon, pi02w SysMon, Node Talker, amp Node Talker, Node Recorder, Pibrary) start with stubbed hardware; instmon, wifimon and 44helper are not among them. The workflow step is still labelled "six servers". Every route found in the source refuses without a login. Health answers; unknown paths, bad logins and a forged cookie are refused; six wrong logins cause a lockout. | Yes (as root) | 72 passed |
| `test_pages.py` | Six pages (both Dashboards, both SysMons, Node Talker, Node Recorder, before and after login) load in headless Chromium with no script or console errors, has the right title and tab set, and each clicked tab shows its panel | Yes | Not run here (needs a root password set for the tests) |
| `test_authed.py` | With a login, on the eight test servers except Pibrary: every route, and "badge truth", meaning what a status badge shows matches the service state that the test sets | Yes | Not run here (same reason) |
| `test_talker_stream.py` | 34 tests of Node Talker: login and events (including the X-Requested-With refusal), WebSocket handshake and origin checks, idle close, listen streams, stalled listeners, silence, and a soak test (skipped unless asked for) | **No.** The baseline suites import it only for its helpers, so its own tests never run in CI | 34 run: 33 passed, 1 skipped (soak), 3.6 minutes |

| | | | |
|---|---|---|---|
| `test_security.py` **(new)** | For each of the eleven servers: the response headers, the request-shape guard on login, the loopback-proxy rule, the Dashboard's local-API key, and (logged in) the guard on a protected POST, wifimon's proxy refusal and the 44helper terminal's proxy refusal | Yes | 25 passed without a login; the logged-in checks run in CI |
| `test_install.py` **(new)** | Every pi02w and common tool installs and uninstalls cleanly: unit file, launcher, command link, and the launcher removed with the last unit | Yes (opt-in, as root) | Passed for all eight tools |
| **Android job** | Unit tests, a release build and lint of the app | Separate workflow | Not run here |

Results come from runs on this branch under Python 3.13 (CI uses 3.11). The Chromium suites set the root password, so they were left to CI. `check_repo.py --base origin/main` passed: 108 changed files.

# From main to Stable

- `main` is the Testing channel (opt-in on nodes). Stable is the default, and only the `promote-stable` workflow writes it.
- The rules are in `channels/rules.json`: a release becomes stable once 4 newer releases of that tool are on main and it has been there 3 days, or once it has been the newest release for 14 days. Cosmetic builds do not count. A floor sets the lowest version a tool may be promoted at.
- `channels/overrides.json` (changed only by pull request) can hold a version back or send a hotfix to Stable at once. The file today holds back versions of wifimon, 44helper, Node Talker and instmon, and hotfixes versions of instmon and pibrary.
- The workflow runs daily and on every push to main, and retries its push up to three times.
- On a node, instmon checks each download against its git blob id, so a changed file is refused (instmon part 1 of this series).

The delay is the audit: a bad release has days on Testing to be noticed, and a hold in `overrides.json` stops it reaching Stable.

# Guards, audits and health readings

## 5.1 Guards that act before a change

| WHERE | GUARD |
|---|---|
| **instmon** | Confirmation on installs, scripts and updates; a syntax check and a self-install check before an install; no action during a Full Update; actions locked while Node Recorder is recording; one install per tool; library locks; a 60-second stay-up watch after an install; Full Update rollback, and a 90-second timer that restores instmon itself. |
| **44helper** | The command a step runs must equal the command in its table; a masked-service gate; a broken-package gate (`dpkg --audit`); field rules; the terminal's switch, address, origin, password and ticket checks; a 15-minute auto-undo on the tunnel tests. |
| **Node Controller** | One link lock for all connects; the bridge reaper and watchdog; a boot disconnect that waits for Asterisk. |
| **Node Talker** | A key-up refused when the node is not ready, not linked or busy; a time-out and a dead-man stop; a load guard that stops tests when the Pi is stressed. |
| **wifimon** | Watchdogs for Wi-Fi, voltage and the tunnel; a test of a new network with undo; a control gate on changes. |

## 5.2 Audits and readings

| READING | WHERE | WHAT IT TELLS YOU |
|---|---|---|
| **Security audit, 14 checks** | Full-build SysMon only | Port exposure, default credentials, SSH, Fail2ban, Monit, service user, file permissions, and more. **The pi02w build has no audit.** |
| **Self-check and readiness** | 44helper | The tunnel interface, `wg0.conf` permissions, firewall exposure, and whether each setup step is really done |
| **Boot report** | 44helper (Boot tab, `--boot-report`) | Uptime, clock sync, power, journal warnings, restarted or never-started units; saved 6 and 25 minutes after each boot |
| **Receive-health verdict** | Node Talker | Which layer lost audio: the network outside the LAN, the Pi, the Pi's Wi-Fi, the link to the listener, or the phone |
| **Audio check, tone test, kerchunk test** | Node Talker | Measured levels, drop-outs, delay and bandwidth; rules that suggest a setting change |
| **Suite Log and coverage** | instmon | A redacted record of journals, files, Wi-Fi and 10-minute snapshots, and a coverage table showing which expected sources produced rows |
| **Quick tests and conflicts** | SysMon | For each pinned service: installed, active, port listening, config present; and duplicate ports |
| **Shutdown reports** | wifimon | The newest 10 reports from watchdog shutdowns |

# How the summaries were checked against the code

Each of the 26 summaries was read against the source. The methods below found real errors, and each can be repeated.

| METHOD | WHAT IT CATCHES | EXAMPLE FROM THIS SERIES |
|---|---|---|
| **Search for the constant or string** | A wrong number, name or path | Lockout starts at the fifth failure, not the first; each tool has its own cookie name |
| **Load the module and count its tables** | Counts that were estimated by eye | The 44helper step tables: ASL3 has 16 install steps, not 15; Ampersand 7 and 6, not 12 |
| **Compare function trees (`ast`)** | A shared function that has drifted | All 52 copies of the 13 shared functions match |
| **Hash extracted text** | A copied block that differs | The launcher is identical in all 11 files |
| **Parse the markup with an HTML parser** | Counts distorted by script text | Control and ARIA counts per page, corrected |
| **Follow a call to its end** | A claim about a feature that the code does not support | Phone tones are sent by the Dashboard, not by Node Talker; the kerchunk log is kept in the Suite Log file |
| **Read the gate, not the label** | A name that sounds like a rule | Pibrary's "access" and SysMon's "lan" flag only label a client; they refuse nothing |
| **Search the whole repo for what is absent** | A missing protection | No sandbox directive in any unit; five tools serve plain HTTP; the pi02w SysMon has no audit |
| **Render and look** | Layout faults | A wrong footer label, and pages holding one stray line |

## 6.1 Kinds of error found

- Numbers and names that were right once and drifted (counts, limits, file names).
- A claim taken from a manual and not checked in code, or the reverse.
- A feature described from its label, when its code did something narrower.
- Something real that the first reading missed (connection caps, the shared login file, the custom command mode).

Summaries mark what was not checked. Those lists are the next place to look.

# A plan, in order

## 7.1 In the repo

1. Run `python3 ci/check_repo.py --base origin/main`. Fix every line it prints.
2. Run `python3 ci/test_ci.py`, then the baseline and server tests. A failure in a baseline means the audio changed.
3. For a page change, run the page and authenticated tests on a runner that has Chromium, or open the page headless and watch the console.
4. Run `pyflakes` and `node --check` on what you touched, and exercise the changed behaviour.
5. Search for the name you changed in every tool: shared functions and the launcher live in many files, and the full build repeats the pi02w code.

## 7.2 On a node

1. Look at the Boot tab or run `44helper.py --boot-report --json`: clock, power, restarted units.
2. Look at SysMon's quick tests and port conflicts, and (full build) the audit.
3. For audio, read Node Talker's health verdict first. It names the layer that lost audio.
4. Start an instmon Suite Log collection, reproduce the fault, and read the coverage table for silent sources.
5. Check the tool's own log and the service journal; look for lockouts, refused requests and rollbacks.

## 7.3 Symptom to place

| SYMPTOM | LOOK AT |
|---|---|
| **A tool will not update or install** | instmon log; the channel and its stable list; a hold in `overrides.json`; a lock or an install already running |
| **A link will not make or drop** | The Dashboard link lock and its "Busy" message; the bridge reaper; Node Talker's guard (it acts only when it owns the node) |
| **Audio drops or is late** | Node Talker's verdict row by row, then the load guard and the Pi's power flags |
| **A key-up is refused** | The reason code: not-ready, not-linked, busy, in-use (TX path) |
| **The node rebooted or shut down** | wifimon: the network-loss action, the voltage watchdog and its shutdown reports |
| **The tunnel is not working** | The tunnel guard's state file, the handshake, the boot order units, and any half or exceptions test still armed |
| **A page will not load or log in** | The lockout (15 minutes or a doubling wait), the shared Dashboard and SysMon session, a proxy header refusal, the connection cap |
| **A setting has no effect** | Node Talker's level switch; whether the tool was restarted (launcher flags are read at start) |

# What is missing, and what to add

Each gap below was checked in the repository. The status column says what was applied in this round; the rules that keep them applied are in the Testing and audit policy in CLAUDE.md.

| # | GAP (EVIDENCE) | FIX | STATUS |
|---|---|---|---|
| 1 | The Node Talker stream tests (34) never run in CI. The workflow has no step for `test_talker_stream.py`. | Add a step to `checks.yml`: `python3 ci/test_talker_stream.py` (about 4 minutes; the soak stays off). | Applied |
| 2 | instmon, wifimon and 44helper have no server tests: no check that their routes refuse without a login, that a wrong login is refused, or that they lock out. | Add the three to `SERVERS` in `ci/test_servers.py` with temporary config folders (instmon reads `INSTMON_CONFIG_DIR` and `INSTMON_LIBRARY_DIR`; wifimon and 44helper use fixed folders, so they need a folder override or the runner's own paths). Then add them to the authenticated and page tests, with Pibrary. | Applied to the server, security and install tests (eleven servers). Page and logged-in tests for the three are still to add |
| 3 | No test sends a request that should be refused for its shape: a POST with no `X-Requested-With` (or, in the Dashboard and SysMon, no JSON type), a request with a proxy header to wifimon or the 44helper terminal, a local-API call without the key or from a non-loopback address, or a WebSocket with a foreign Origin (only Node Talker's is tested). | A new `ci/test_security.py` driven by a table of expected answers per tool: 403 or 415 for the missing header or type; 403 for a proxy header; 403 for the local API without the key. Run it against the same stub servers. | Applied (`test_security.py`). Found: instmon and Pibrary take a login without the header; recorded as a known gap |
| 4 | Response headers are not tested. The security summary lists which tool sends which header, but nothing keeps it true. | In `ci/test_security.py`, fetch each page and compare `X-Frame-Options`, `nosniff`, `Referrer-Policy` and `Content-Security-Policy` with a table, so a missing header fails CI. | Applied |
| 5 | The workflow runs on pull requests only, not on pushes to main, so the merged result is never tested. | Add `push: branches: [main]` to `checks.yml`. | Applied |
| 6 | CI uses Python 3.11 only. Debian 13 (Trixie) nodes run 3.13. | Run the Python steps on a matrix of 3.11 and 3.13. (The suites passed on 3.13 when run for this summary.) | Applied (matrix 3.11 and 3.13) |
| 7 | 44helper's exact-command rule, its field rules and its custom mode have no tests. A step field with no rule makes the step unrunnable, and nothing finds that. | Unit tests in `ci/test_ci.py`: for every step table, every field has a rule in `_STEP_FIELD_RULES`; `_step_command` builds the shown command; a changed command is refused; a bad field value is refused. | Applied (8 tests) |

| | | | |
|---|---|---|---|
| **8** | The comment-stripped rule is not enforced. Today no tool file has a Python comment or docstring (checked with the tokenizer), but `check_repo.py` would not notice one. | In `check_python`, fail on any comment token after the shebang and coding lines, and on any docstring; for shell, allow only the shebang and title line. | Applied (Python and shell) |
| **9** | The launcher and `-OO` are never exercised. The test servers start each tool directly. | A test that writes the launcher from a tool file, runs a small target through it, and checks the re-run keeps the process ID, applies `-OO` and the arena cap, removes the cap from child processes, honours each `launch_no_*` flag, and loads a file whose name starts with a digit. Also start one server-test run through the launcher. | Applied (tests, and servers start through the launcher in CI) |
| **10** | `--install` and `--uninstall` are untested: unit files, links, the launcher, firewall ports, and the rule that the launcher is removed only when no unit uses it. | A test that runs each tool's install and uninstall in a temporary root with `systemctl`, `firewall-cmd` and `ufw` replaced by stubs on `PATH`, and checks what was written and removed. | Applied (`test_install.py`) |
| **11** | Promotion to Stable does not check that the commit passed CI. `ci/promote.py` reads only the history of main. | Make `promote.yml` skip a release whose commit has no successful `checks` run, or rely on branch protection and say so in the README. | Applied (promotion waits for green checks) |
| **12** | Shell scripts get only `bash -n`. | Run `shellcheck -S error` on changed `.sh` files in `check_repo.py`. | Applied (CI requires shellcheck) |
| **13** | Only changed files are checked. An unchanged tool is not re-checked when Python or Node changes. | A weekly scheduled run that checks every tool file, not only the changed ones. | Applied (weekly `--all` run) |
| **14** | Disk-image backup, restore, clone and format are not tested. instmon accepts only USB or removable drives, so a loop device cannot stand in. | Add a test-only switch (as Pibrary has `PIBRARY_TEST_ALLOW_LOOP`) and test the jobs on loop devices made from image files. | Not applied: needs an instmon release with a test switch |
| **15** | Unused code is not found. Two page-script functions in 44helper are never called. | A warning in `check_repo.py` for page-script functions defined and never named, with an allow-list for names built at run time. | Applied as a note; removal goes in 44helper's next release |
| **16** | The summaries drift from the code. Each names its source file in the footer, and nothing warns when that file is replaced. | A note in `check_repo.py`: when a tool gets a new file name, list the files in `docs/src` that still name the old one. | Applied as a note |
| **17** | No test runs on a real node. Asterisk, DVSwitch, WireGuard, firewalld and the radio path are stubbed. | A short release checklist run on a Pi Zero 2 W before a hold is lifted: link and unlink each mode, a tone test, a tunnel restart, a reboot, and the boot report. | Added to the policy |
| **18** | The workflow step says "six servers". There are eight. | Correct the label in `checks.yml`. | Applied |

## 8.1 Product gaps the tests cannot fix

- The pi02w SysMon has no security audit. Only the full build does.

- Five tools serve plain HTTP, and every service runs as root without sandboxing (see the security summary).
- Memory and timing on a Pi Zero 2 W (the launcher savers, the light-theme filter) were not measured.

## 8.2 Order used

First the workflow-only changes (1, 5, 6, 18); then the server and security tests (2, 3, 4), which cover the largest untested surface; then the small additions (7, 8); then 9 to 12 and the rest.

# One round of audit and fixes, step by step

This is how the gaps in Part 8 were found and fixed. It is an example of the method in Parts 6 and 7, applied to the tests themselves. Each step names what was run and what it showed.

| STEP | WHAT WAS DONE | WHAT IT SHOWED |
|---|---|---|
| **1. Check the document against the repo** | Read `ci/test_servers.py`, `ci/test_pages.py`, `ci/test_authed.py` and the workflow, and compared them with this summary. | Three errors in this summary: the server tests covered eight servers, not six; the stream tests were not run by CI at all; the page tests covered six pages. |
| **2. Prove each gap before listing it** | Searched `ci/` for each behaviour (proxy headers, local key, headers, launcher, install). Counted comments with the Python tokenizer, not with a text search. | 18 gaps, each with evidence. A text search had counted `#` inside strings as comments; the tokenizer found none. |
| **3. Run what CI skipped** | `python3 ci/test_talker_stream.py` | 34 tests, 33 passed and the soak skipped. Safe to add to CI. |
| **4. Add the missing servers and read every failure** | Added instmon, wifimon and 44helper to the server harness, with stubs so a reboot or power-off is only logged. | Seven failures, none a security fault: the lockout test ran first and locked later tests out; these tools reply with `error` or `output`, not `message`; two wifimon routes and one instmon page are public on purpose. The harness was fixed; 99 tests passed. |
| **5. Probe before writing expectations** | A short script asked every server the same questions: headers, login without the CSRF header, a proxy header, the local API without a key. | A real finding: instmon and Pibrary take a login without `X-Requested-With`. Recorded as a known gap, not hidden. |
| **6. Pin the answers in a test** | Wrote `ci/test_security.py` from the probe results. | 25 checks pass without a login. The logged-in checks need a root password, which this container could not set, so they run in CI. |
| **7. Check what could not be run** | Read each logged-in check against the tool's POST handler. | One expectation was wrong: SysMon has no `/api/action`; the check now uses `/api/config`. |
| **8. Tighten the repo checker** | Added the comment, docstring and shellcheck rules, `--all`, and two notes; wrote self-tests for each; ran it on every script in the repo. | 139 files pass. The unused-function note found exactly the two known functions. shellcheck (installed in a scratch environment) passed all four shell scripts. |
| **9. Test the launcher** | Ran a small target through the launcher text taken from a tool, then ran all servers through it. | Process ID kept, `-OO` on, arena cap kept from children, switches honoured, digit-first names loaded. 99 server tests passed through the launcher. |
| **10. Test install and uninstall** | Installed and removed all eight tools in this disposable container, then looked for leftovers. | All passed in 6 seconds; nothing was left in the unit, binary or library folders. |

| | | |
|---|---|---|
| **11. Change the workflows and re-read them** | Added the triggers, the Python matrix and the new steps; added a promotion gate. | The first gate would have blocked forever (the bot's own `stable.json` commit has no checks run). Fixed to look at the newest non-`stable.json` commit. |
| **12. Write the rule down** | Added the Testing and audit policy to CLAUDE.md. | The fixes stay applied: new tools join the tests, the security table is a contract, known gaps are listed until fixed. |

## 9.1 Results of the round

| SUITE | BEFORE | AFTER |
|---|---|---|
| **Self-tests ( `test_ci.py` )** | 241 | 259, all passing |
| **Server tests** | 8 servers, 72 tests | 11 servers, 99 tests, also through the launcher |
| **Security tests** | none | 77 cases (25 without a login, the rest in CI) |
| **Install tests** | none | 8 tools |
| **Stream tests in CI** | not run | 34 run |
| **Python versions in CI** | 3.11 | 3.11 and 3.13 |

## 9.2 What the example teaches

- **A failing new test is a question, not a verdict.** Most failures in step 4 were in the test, not the tools.
- **Probe, then pin.** Asking the real servers first gave a table that matched the code, and surfaced a real gap.
- **Say what was not run.** The logged-in checks were checked by reading, and that is stated, until CI runs them.
- **Re-read your own fix.** The promotion gate was wrong on the first try and was caught by thinking through the next run.